

Feature Transformation and Engineering



Table of Contents

#	Topic	Subtopics
1	Feature Scaling	Standardization, Min-Max, Robust Scaling
2	Feature Encoding	One-Hot, Label, Target, Frequency, Binary, Rare Categories
3	Discretization & Binning	<code>pd.cut()</code> , <code>pd.qcut()</code>
4	Feature Interactions / Engineering	Arithmetic, Statistical Aggregations, Boolean Combinations
5	Feature Selection	Filter Methods, Wrapper Methods, Permutation Importance
6	Data Cleaning Log Report	Class-based Logging, Exports

Topic 1: Feature Scaling

"When you have different orders of magnitude within your data, you must normalize them."

Machine learning algorithms that use **distance metrics** — such as k-nearest neighbors, support vector machines, and neural networks — are particularly sensitive to the scale of input features. Feature scaling **transforms numerical features to a common scale**.

Key Benefits of Feature Scaling

1. **Algorithm Convergence** — Gradient descent-based algorithms converge faster when features are on similar scales.
2. **Preventing Feature Dominance** — Ensures no single feature dominates due to scale differences, making feature importance more interpretable.
3. **Numerical Stability** — Reduces risk of numerical overflow or underflow in calculations.
4. **Improved Model Performance** — Many algorithms perform better with scaled features.

 **Critical Rule:** Always `fit` the scaler on **training data only**, then `transform` both train and test sets!



1.1 Standardization (Z-Score Normalization)

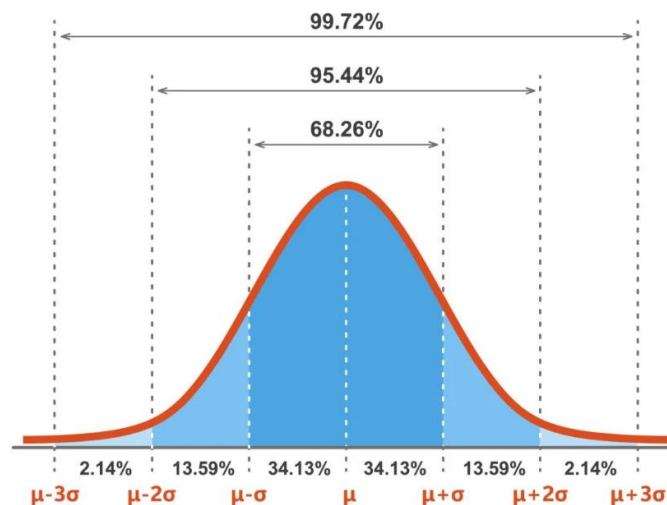
Goal: Transform data to have a **mean of 0** and **standard deviation of 1**, creating a "standard normal distribution."

Formula
$$z = \frac{x - \mu}{\sigma}$$

- μ = mean of the feature
- σ = standard deviation of the feature

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['age_std', 'salary_std']] = scaler.fit_transform(df[['age', 'salary']])
```

What happens: Calculates the mean and standard deviation for each column, then applies the formula to every value.



Key Properties

Property	Value
Output Mean	0
Output Std Dev	1
Range	Unbounded (no fixed min/max)
Sensitive to Outliers	Moderate
Preserves Distribution Shape	Yes ← very important!

🔑 `StandardScaler` does **NOT** change the shape of the distribution — only rescales it. If your data was skewed before, it stays skewed after standardization!

Best For

- Algorithms that **assume data is centered and normally distributed**: linear regression, logistic regression, PCA, SVM

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()  
df1[['Age_zscore', 'Salary_zscore']] = scaler.fit_transform(df1[['Age', 'Salary']])  
df1 = df1.round(4)
```

=====

Example 1: Age vs Salary Standardization

=====

Name	Age	Salary	Age_zscore	Salary_zscore
Alice	25	35000	-1.1359	-1.0436
Bob	45	90000	0.6671	0.6749
Carol	35	55000	-0.2344	-0.4187
David	28	42000	-0.8655	-0.8249
Eve	55	120000	1.5687	1.6123

Age → Mean: 37.60, Std: 11.09

Age_zscore → Mean: 0.00, Std: 1.00

Salary → Mean: 68400.00, Std: 32003.75

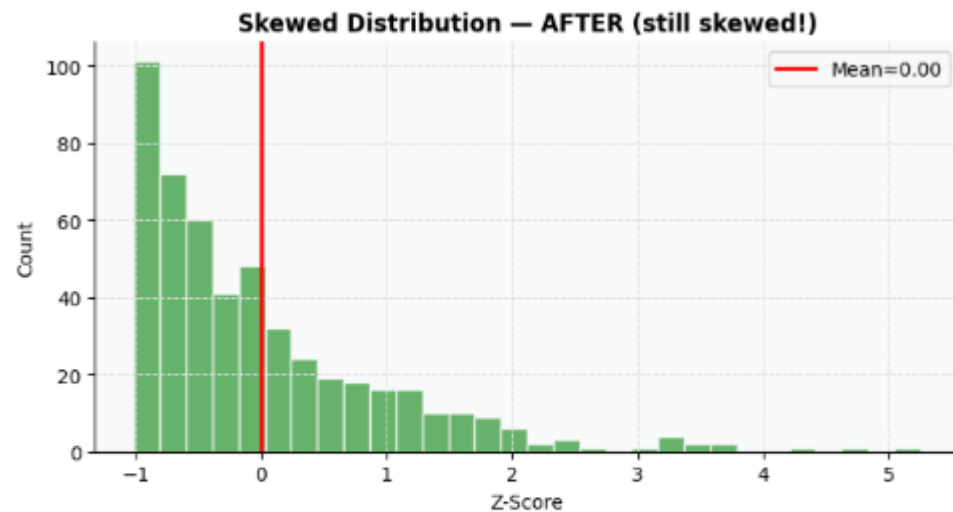
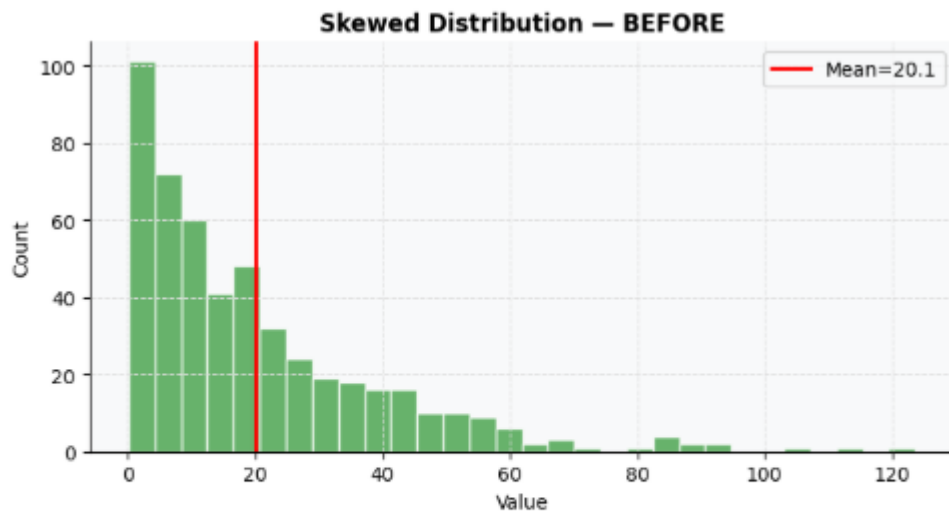
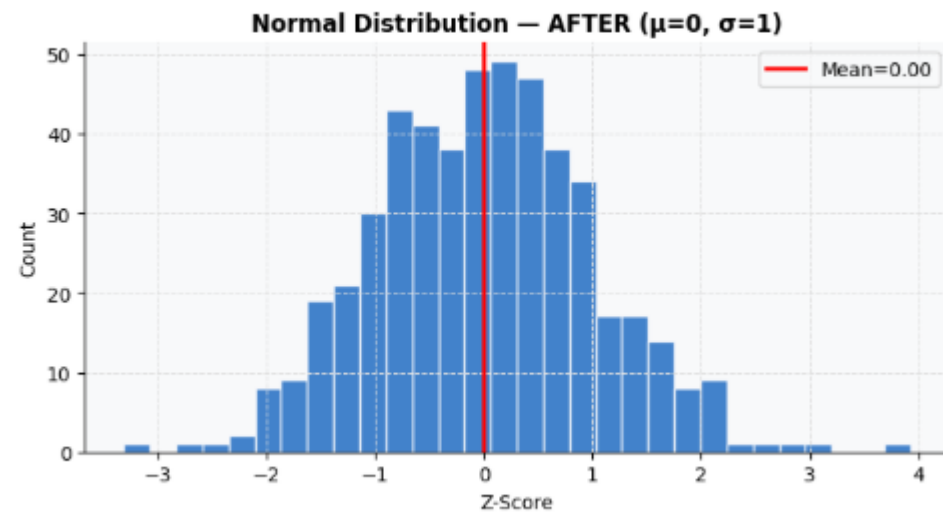
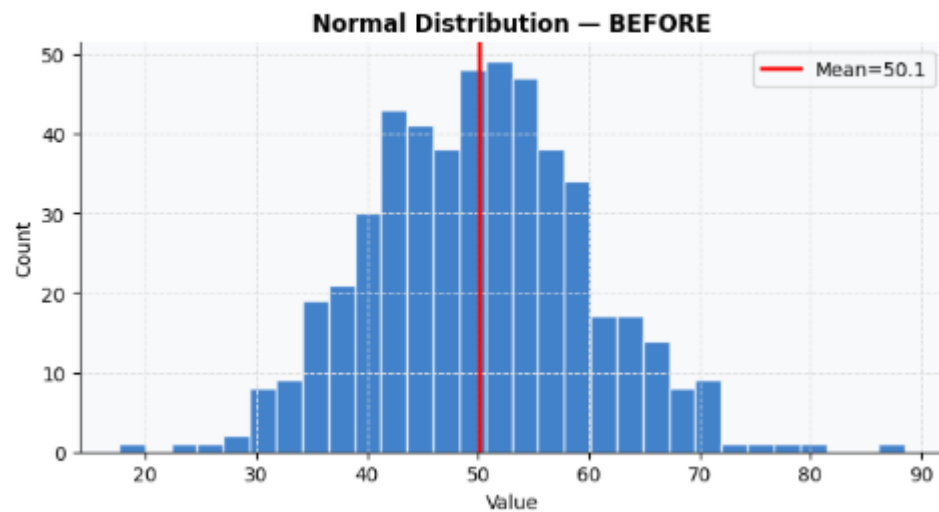
Salary_zscore → Mean: 0.00, Std: 1.00

Example 2: Effect of Outlier on Standardization

Score	Score_std
50	-0.4758
55	-0.3854
52	-0.4396
48	-0.5119
53	-0.4216
200	2.2343

Original mean = 76.33, std = 55.35

Standardization: Distribution Shape is PRESERVED



✓ Key Insight: Standardization shifts and scales but NEVER changes the shape!



1.2 Min-Max Normalization

Goal: Rescale features to a **fixed range** [0, 1].

Formula

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['age_norm', 'salary_norm']] = scaler.fit_transform(df[['age',
'salary']])
```


Key Properties

Property	Value
Output Range	[0, 1] (default)
Minimum value →	0
Maximum value →	1
Sensitive to Outliers	HIGH ⚠

⚠ **Outlier Warning:** If an extreme outlier is the max, all other values get compressed into a tiny sub-range near 0. Use with caution on data with outliers!

Best For

- Algorithms that rely on **distances or gradients**: neural networks, k-means, k-NN
- When features have **different units/scales** but you want them comparable
- When data has **no significant outliers**

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler_mm = MinMaxScaler()  
normalized = scaler_mm.fit_transform(df_mm_vals) # Out: <class 'numpy.ndarray'>  
df_norm = pd.DataFrame(normalized, columns=df_mm_vals.columns).round(4)
```

BEFORE:

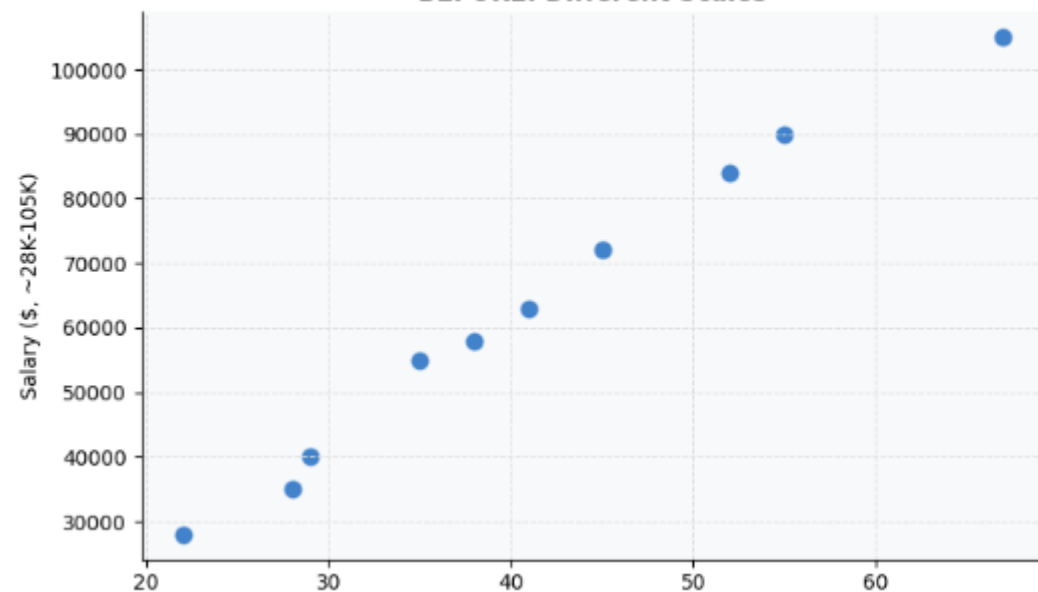
Temperature	Pressure	Humidity	Wind
10	99.0	50	1.0
15	103.0	40	2.0
20	101.0	60	3.0
25	101.3	68	5.2
30	100.0	75	7.0
35	103.0	80	4.0

AFTER (all values in [0, 1]):

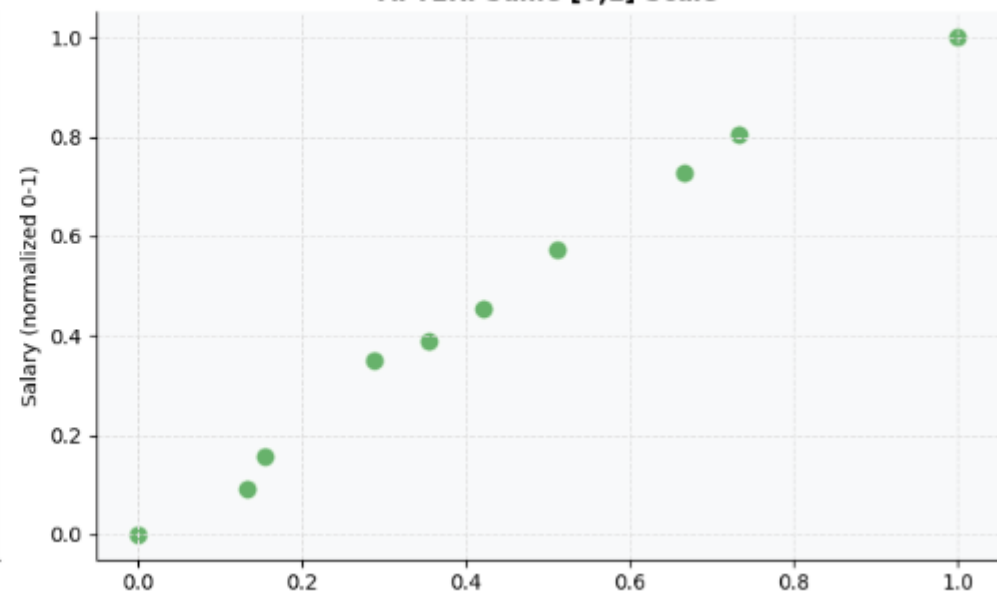
Temperature	Pressure	Humidity	Wind
0.0	0.000	0.250	0.0000
0.2	1.000	0.000	0.1667
0.4	0.500	0.500	0.3333
0.6	0.575	0.700	0.7000
0.8	0.250	0.875	1.0000
1.0	1.000	1.000	0.5000

Min-Max Normalization: Before vs After

BEFORE: Different Scales

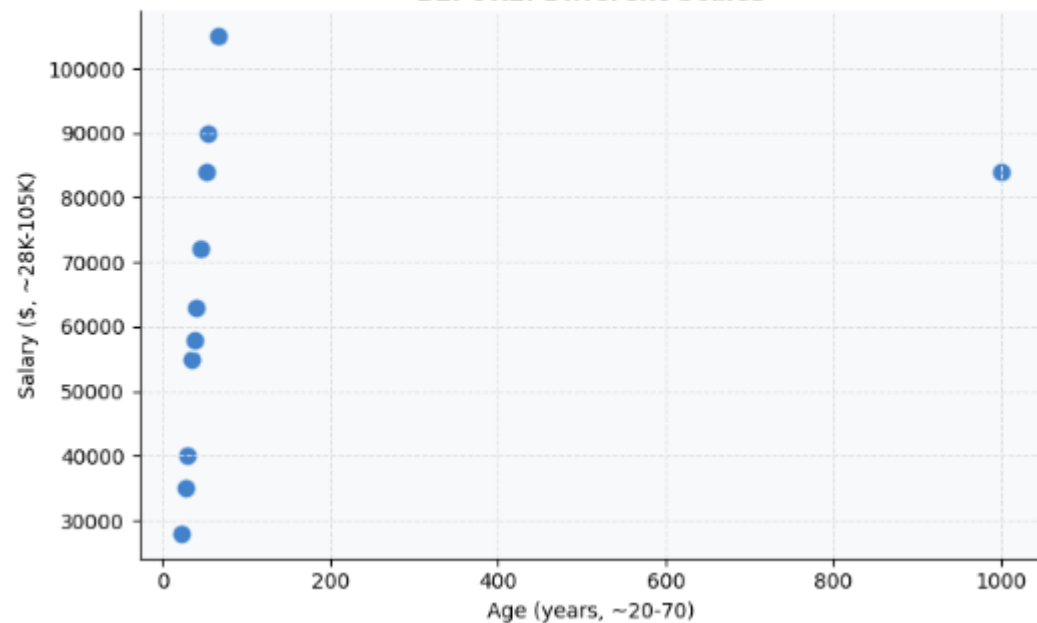


AFTER: Same [0,1] Scale

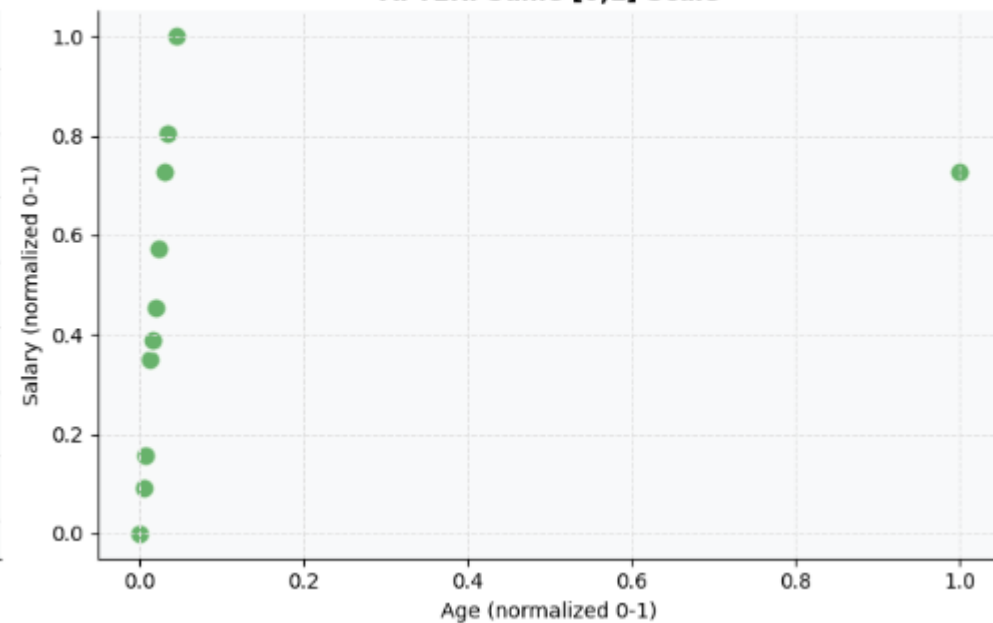


Min-Max Normalization: Before vs After

BEFORE: Different Scales



AFTER: Same [0,1] Scale





1.3 Robust Scaling

Goal: Like Z-score, but uses **median** and **Interquartile Range (IQR)** instead of mean and std, making it **resistant to outliers**.

Formula

$$x_{robust} = \frac{x - \text{median}}{\text{IQR}} \quad \text{where} \quad \text{IQR} = Q_3 - Q_1$$

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df[['age_r', 'salary_r']] = scaler.fit_transform(df[['age', 'salary']])
```

Key Properties

Property	Value
Range	Unbounded
Centers at	Median
Scales by	IQR
Sensitive to Outliers	LOW ✓
Preserves Shape	Yes



Best For

- Data **with outliers** that you cannot remove
- Keeps distribution shape but reduces outlier influence

```
from sklearn.preprocessing import RobustScaler, StandardScaler, MinMaxScaler
```

```
std_scaler = StandardScaler()  
mm_scaler = MinMaxScaler()  
rob_scaler = RobustScaler()
```

```
data_with_outliers['Z-Score'] = std_scaler.fit_transform(data_with_outliers[['Salary']])  
data_with_outliers['Min-Max'] = mm_scaler.fit_transform(data_with_outliers[['Salary']])  
data_with_outliers['Robust'] = rob_scaler.fit_transform(data_with_outliers[['Salary']])  
data_with_outliers = data_with_outliers.round(4)
```

Comparison of Scaling Methods on Data WITH Outlier:

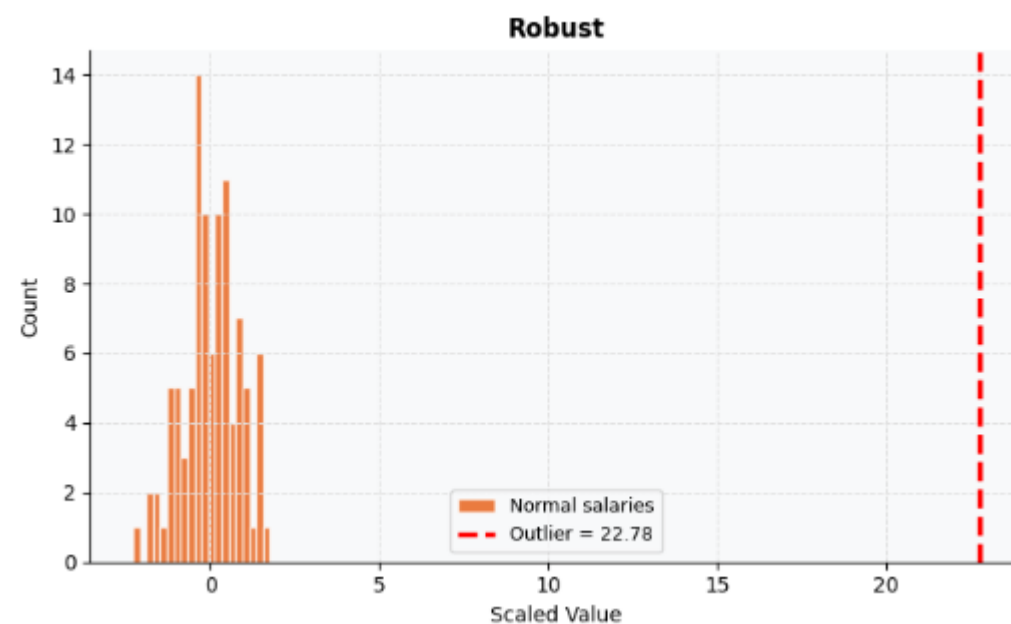
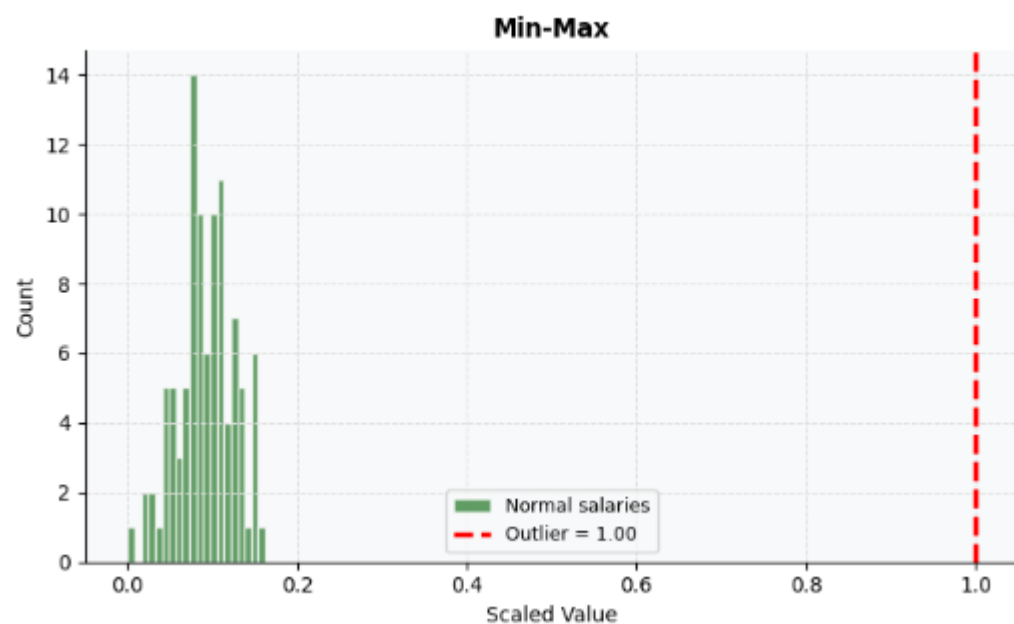
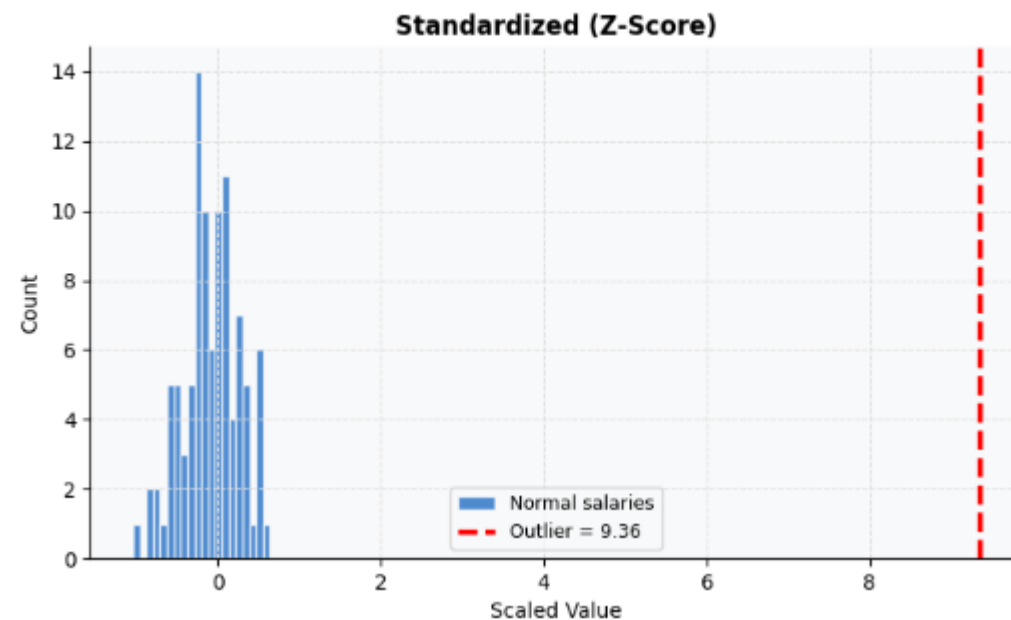
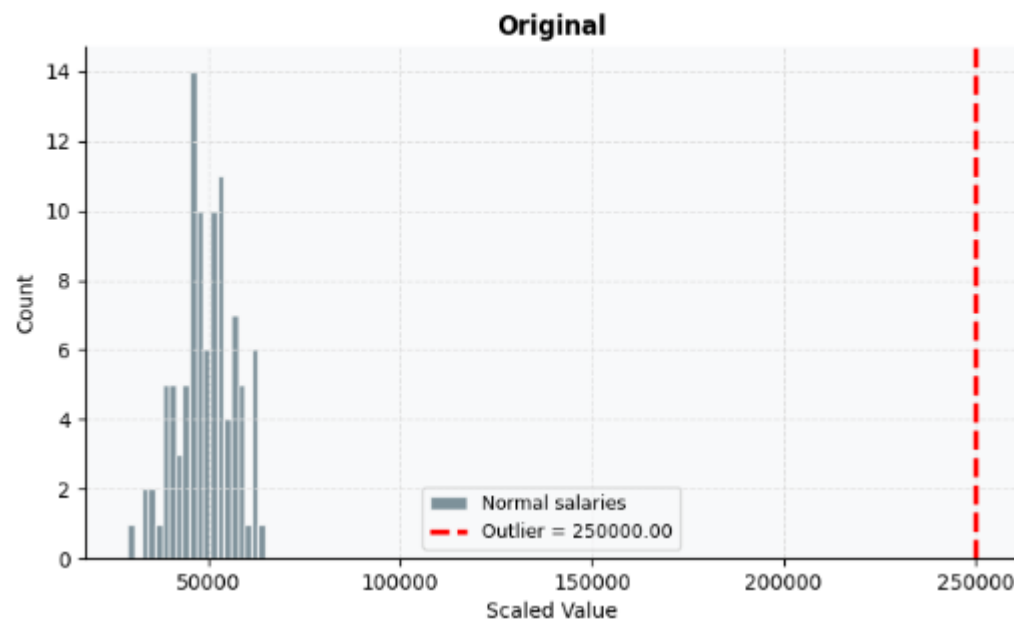
=====

Salary	Z-Score	Min-Max	Robust
30000	-0.4011	0.0000	-0.8421
35000	-0.3687	0.0106	0.2105
32000	-0.3881	0.0043	-0.4211
38000	-0.3492	0.0170	0.8421
31000	-0.3946	0.0021	-0.6316
36000	-0.3622	0.0128	0.4211
33000	-0.3816	0.0064	-0.2105
500000	2.6454	1.0000	98.1053

Stats of all Columns:

	Salary	Z-Score	Min-Max	Robust
count	8.00	8.00	8.00	8.00
mean	91875.00	-0.00	0.13	12.18
std	164928.93	1.07	0.35	34.72
min	30000.00	-0.40	0.00	-0.84
25%	31750.00	-0.39	0.00	-0.47
50%	34000.00	-0.38	0.01	0.00
75%	36500.00	-0.36	0.01	0.53
max	500000.00	2.65	1.00	98.11
IQR	4750.00	0.03	0.01	1.00

Salary Distribution: 99 Normal + 1 Outlier (\$250K) Effect of Different Scaling Methods





Feature Scaling — Comparison Summary

Method	The Zero	Scales by	Range	Outlier Sensitivity	Use Case
StandardScaler	Mean (0)	Std Dev (1)	Unbounded	Moderate	Normal distributions, distance-based algorithms
MinMaxScaler	Min (0)	Max - Min	[0, 1]	High ⚠️	Bounded ranges, neural networks
RobustScaler	Median (0)	IQR (1)	Unbounded	Low ✅	Data with outliers

⚠️ **Golden Rule: Always `fit` on training data only, then `transform` both train and test!**

```
scaler.fit(X_train)           # Learn parameters from train only
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)  # Use same parameters!
```

Does the output of robust scalar always have Median = 0 and IQR = 1?

Yes, always. That's exactly what RobustScaler does by design:

```
scaled = (x - median) / IQR
```

- Subtracting **median** → shifts median to **0**
- Dividing by **IQR** → scales IQR to **1**

So after scaling, **median** = **0** and **IQR** = **1** are guaranteed, always.

Similarly, The output of **standard scalar (z-score)** has **Mean** = **0** and **Std** = **1**.



Topic 2: Feature Encoding

"Many machine learning algorithms can support categorical values without further manipulation but there are many more algorithms that do not."

Feature Encoding converts discrete categories or groups into **numerical form** so that machine learning algorithms can process them.

Types of Categorical Variables

Type	Description	Examples
Nominal	No inherent order	Color (red, blue, green), city names
Ordinal	Natural ordering exists	Education (HS < BS < MS < PhD), satisfaction (low/medium/high)
Binary	Only two categories	gender (M/F), yes/no



2.1 One-Hot Encoding

Creates binary columns (0/1) for each unique category.

For `k` unique categories, one-hot encoding creates `k-1` new binary columns (when `drop_first=True`).

```
df_encoded = pd.get_dummies(df, columns=['Color', 'Size'], drop_first=True)
```



Pros and



Cons

 Pros	 Cons
No artificial ordering	Curse of dimensionality (many categories = many columns)
Works with all algorithms	Memory-intensive for high cardinality



Best For: Nominal variables, low cardinality (< 10–15 categories), linear models, neural networks



Use `drop_first=True` to avoid multicollinearity (dummy variable trap)!

In pandas, `drop_first=True` drops the first category alphabetically.

```
# With drop_first (k-1 columns – avoids multicollinearity)
df_ohe = pd.get_dummies(df_color, columns=['Color', 'Size'], drop_first=True, dtype=int)
```

Original DataFrame:

Product	Color	Size
Shirt A	Red	S
Shirt B	Blue	M
Shirt C	Green	L
Shirt D	Red	M
Shirt E	Blue	S

After One-Hot Encoding (drop_first=False – ALL categories):

Product	Color_Blue	Color_Green	Color_Red	Size_L	Size_M	Size_S
Shirt A	0	0	1	0	0	1
Shirt B	1	0	0	0	1	0
Shirt C	0	1	0	1	0	0
Shirt D	0	0	1	0	1	0
Shirt E	1	0	0	0	0	1

After One-Hot Encoding (drop_first=True – avoids multicollinearity):

Product	Color_Green	Color_Red	Size_M	Size_S
Shirt A	0	1	0	1
Shirt B	0	0	1	0
Shirt C	1	0	0	0
Shirt D	0	1	1	0
Shirt E	0	0	0	1



'First of Color and Size alphabetically (Blue, L) are the DROPPED reference categories



2.2 Label Encoding

Assigns a unique integer to each category.

Simpler than one-hot encoding but can introduce **unintended ordinal relationships**.

```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
df['education_encoded'] = le.fit_transform(df['education'])
```



When to Use Label Encoding

1. **Tree-based algorithms:** Decision Trees, Random Forests, Gradient Boosting
2. **Ordinal variables:** When natural ordering exists and should be preserved
3. **High cardinality:** When one-hot encoding would create too many columns

⚠ Danger: Label encoding can make linear models interpret Red=0, Blue=1, Green=2 as "Green > Blue > Red", which has no real meaning!

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()  
df_edu['Education_encoded'] = le.fit_transform(df_edu['Education'])
```

```
print(f"\nEncoding: {dict(zip(le.classes_, le.transform(le.classes_).tolist()))}")
```

LabelEncoder Encoding (alphabetical order):

Student	Education	Education_encoded
Ali	High School	1
Mona	Bachelor	0
Karim	Master	2
Sara	PhD	3
Omar	Bachelor	0
Laila	Master	2

Encoding: {'Bachelor': 0, 'High School': 1, 'Master': 2, 'PhD': 3}

Order: Bachelor(0) < High School(1) < Master(2) < PhD(3) – alphabetical, NOT meaningful!

Is there a way to pass the mapping to `LabelEncoder`?

No, `LabelEncoder` does **not** accept a custom mapping — it always uses alphabetical order. You can't pass an order to it.

Your options:

- `map()` — simplest, full control  (what your original code does)
- `OrdinalEncoder` — accepts custom category order:

```
from sklearn.preprocessing import OrdinalEncoder
```

```
oe = OrdinalEncoder(categories=[['High School', 'Bachelor', 'Master', 'PhD']])  
df_edu['Education_encoded'] = oe.fit_transform(df_edu[['Education']])
```

`OrdinalEncoder` is the sklearn way to pass a custom order.

```
# For ordinal, we want custom mapping (not alphabetical!)
edu_order = {'High School': 0, 'Bachelor': 1, 'Master': 2, 'PhD': 3}
df_edu['Education_encoded'] = df_edu['Education'].map(edu_order)
```

✅ CORRECT: Ordinal Encoding (preserves meaningful order)

Student	Education	Education_encoded
Ali	High School	0
Mona	Bachelor	1
Karim	Master	2
Sara	PhD	3
Omar	Bachelor	1
Laila	Master	2

Order: High School(0) < Bachelor(1) < Master(2) < PhD(3) – meaningful!

```
from sklearn.preprocessing import OrdinalEncoder
```

```
oe = OrdinalEncoder(categories=[['High School', 'Bachelor', 'Master', 'PhD']])  
df_edu['Education_encoded'] = oe.fit_transform(df_edu[['Education']]).astype(int)
```

```
print(f"\nCategories: {dict(enumerate(oe.categories_[0]))}")
```

✅ CORRECT: Ordinal Encoding (preserves meaningful order)

Student	Education	Education_encoded
---------	-----------	-------------------

Ali	High School	0
Mona	Bachelor	1
Karim	Master	2
Sara	PhD	3
Omar	Bachelor	1
Laila	Master	2

Categories: {0: 'High School', 1: 'Bachelor', 2: 'Master', 3: 'PhD'}

Order: High School(0) < Bachelor(1) < Master(2) < PhD(3) – meaningful!

Side Notes

1. LabelEncoder vs OrdinalEncoder

	LabelEncoder	OrdinalEncoder
Input	1D (Series)	2D (DataFrame)
Custom order	✗ always alphabetical	✓ via categories=
Output	1D array	2D array
Use for	target/label column	feature columns

2. .classes_ vs .categories_

	.classes_	.categories_
Belongs to	LabelEncoder	OrdinalEncoder
Type	1D array	list of arrays (one per feature)
Access	le.classes_	oe.categories_[0] (for first feature)

3. Getting {label: encoding} dict

LabelEncoder:

```
dict(zip(le.classes_, le.transform(le.classes_).tolist()))  
# {'Bachelor': 0, 'High School': 1, 'Master': 2, 'PhD': 3}
```

OrdinalEncoder:

```
dict(enumerate(oe.categories_[0]))  
# {0: 'High School', 1: 'Bachelor', 2: 'Master', 3: 'PhD'}
```

4. df['col'] vs df[['col']]

	df['col']	df[['col']]
Type	Series	DataFrame
Dimensions	1D (n,)	2D (n, 1)
Use for	most pandas ops	sklearn (requires 2D input)

2.3 Target Encoding (Mean Encoding)

Replaces categorical values with the *mean of the target variable* for that category.

Powerful but requires careful implementation to avoid **data leakage**!

Example

City	Sales	Target-Encoded City
New York	50000	52500 ← mean of all NY sales
New York	55000	52500
Boston	30000	31000 ← mean of all Boston sales
Boston	32000	31000

Best For

- High cardinality features
- Strong relationship between category and target

Data Leakage Warning

Must use cross-validation or a held-out set — computing the mean on the full dataset leaks target information!

```
# Calculate mean Sales per City
city_means = df_city.groupby('City')['Sales'].mean()
```

```
df_city['City_target_encoded'] = df_city['City'].map(city_means) # Use map()
```

Step 1 – Mean Sales per City:		Step 2 – Replace City with its mean:	
	Sales	City	Sales
City			City_target_encoded
Boston	31000.000000	New York	50000
Chicago	46000.000000	New York	55000
New York	52333.333333	Boston	30000
		Boston	32000
		Chicago	45000
		Chicago	47000
		New York	52000

- `groupby('City')` — groups all rows by city name
- `['Sales'].mean()` — for each group, calculates the mean of the Sales column → returns a Series with City as index and mean Sales as value
- `city_means` — holds `{'Boston': 31000, 'Chicago': 46000, 'New York': 52333}`
- `.to_frame()` — converts the Series to a DataFrame for prettier printing (Series prints as a single column without a header, DataFrame prints with a proper column name `Sales`)
- `.map(city_means)` — replaces each City string with its corresponding mean value from `city_mean`
- `df_city['City_target_encoded']` — stores the encoded numeric column, where each city is now represented by its average Sales instead of its name



2.4 Frequency Encoding

Replaces each category with its *frequency (proportion)* in the dataset.

Preserves information about category importance and handles high-cardinality features well.

```
freq_map = df['City'].value_counts(normalize=True) # proportion
df['City_freq'] = df['City'].map(freq_map)
```

Frequency Map:

```
<class 'pandas.Series'>
```

City

Cairo	0.62
-------	------

Alexandria	0.21
------------	------

Giza	0.17
------	------

Sample of Frequency-Encoded DataFrame:

```
<class 'pandas.DataFrame'>
```

City	City_freq
Cairo	0.62
Alexandria	0.21
Giza	0.17

💡 Frequency (proportion) is more robust across datasets of different sizes.

✅ Frequency encoding is the FREQUENCY itself used as a numeric feature.



2.5 Binary Encoding

More compact than one-hot encoding. Uses $\log_2(n)$ columns instead of n columns. Converts label integers to binary and splits each binary digit into a separate column.

Steps

1. Assign integer labels (Red=0, Green=1, Blue=2, Yellow=3)
2. Convert to binary (0=00, 1=01, 2=10, 3=11)
3. Split binary digits into separate columns

```
df_binary = pd.DataFrame({'Color': colors})
df_binary['Label'] = range(len(colors))
df_binary['Binary'] = df_binary['Label'].apply(lambda x: format(x, '03b'))
df_binary['Bit_2'] = df_binary['Label'].apply(lambda x: (x >> 2) & 1)
df_binary['Bit_1'] = df_binary['Label'].apply(lambda x: (x >> 1) & 1)
df_binary['Bit_0'] = df_binary['Label'].apply(lambda x: x & 1)
```

Binary Encoding – 8 colors using only 3 columns!

Color	Label	Binary	Bit_2	Bit_1	Bit_0
Red	0	000	0	0	0
Green	1	001	0	0	1
Blue	2	010	0	1	0
Yellow	3	011	0	1	1
Purple	4	100	1	0	0
Orange	5	101	1	0	1
Pink	6	110	1	1	0
Black	7	111	1	1	1

✓ 8 categories → 3 binary columns ($\log_2(8) = 3$)
vs One-Hot → 8 columns!

Explanation:

- `range(len(colors))` — assigns a numeric label 0–7 to each color (Red=0, Green=1, ..., Black=7)
- `format(x, '03b')` — converts the integer to a binary string of 3 digits e.g. `5` → `'101'`, `'03b'` means 3 digits, zero-padded, `format(2, '03b')` → `'010'` (zero-padded to 3 digits)
- `(x >> 2) & 1` — right-shifts `x` by 2 bits then masks the last bit → extracts the **leftmost** (most significant) bit
- `(x >> 1) & 1` — right-shifts `x` by 1 bit then masks the last bit → extracts the **middle** bit
- `x & 1` — masks the last bit directly → extracts the **rightmost** (least significant) bit
- `>>` — bitwise right shift: moves bits to the right, dropping the rightmost bits e.g. `5 (101) >> 1 = 2 (010)`
- `& 1` — bitwise AND with 1: keeps only the last bit, always returns 0 or 1 e.g. `6 (110) & 1 = 0`, `5 (101) & 1 = 1`
- `math.ceil(math.log2(n))` — calculates minimum number of bits needed to represent `n` categories e.g. $\log_2(8) = 3$, $\log_2(10) = 3.32 \rightarrow \text{ceil} \rightarrow 4$

```
df_binary = pd.DataFrame({'Color': colors})
df_binary['Label'] = range(len(colors))
df_binary['Binary'] = df_binary['Label'].apply(lambda x: format(x, '03b'))
df_binary['Bit_2'] = df_binary['Label'].apply(lambda x: (x >> 2) & 1)
df_binary['Bit_1'] = df_binary['Label'].apply(lambda x: (x >> 1) & 1)
df_binary['Bit_0'] = df_binary['Label'].apply(lambda x: x & 1)
```



2.6 Handling Rare Categories

Group infrequent categories into an "Other" bucket.

Rare categories can overfit and add noise without adding signal.

Strategies

1. **Threshold-based:** Categories appearing $< 1\%$ (or some threshold) $\rightarrow$ 'Other'
2. **Top-N categories:** Keep only the N most frequent categories
3. **Benefits:** Reduces dimensionality, improves model generalization

```
freq_counts = df_rare['Product'].value_counts()
```

```
# Strategy 1: Threshold-based (< 5% = rare)
threshold = 0.05
freq_pct = freq_counts / len(df_rare)
rare_categories = freq_pct[freq_pct < threshold].index.tolist()
```

```
df_rare['Product_grouped_threshold'] = df_rare['Product'].apply(
    lambda x: x if x not in rare_categories else 'Other'
)
```

```
freq_counts = df_rare['Product'].value_counts()
```

```
# Strategy 2: Keep Top-3 only
top3 = freq_counts.head(3).index.tolist()
# NOTE: output of value_counts() is already sorted desc by count
df_rare['Product_top3'] = df_rare['Product'].apply(
    lambda x: x if x in top3 else 'Other'
)
```

Original value counts:

	Product(freq)	Product(%)
--	---------------	------------

Product		
Laptop	400	40.0
Phone	300	30.0
Tablet	150	15.0
Camera	50	5.0
Smartwatch	40	4.0
Drone	30	3.0
VR_Headset	15	1.5
Robot	10	1.0
Gadget	5	0.5

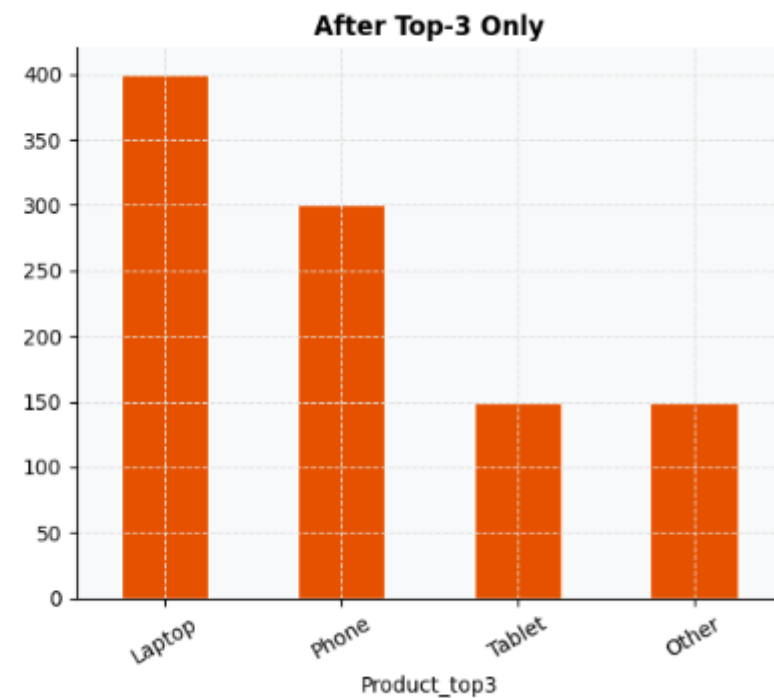
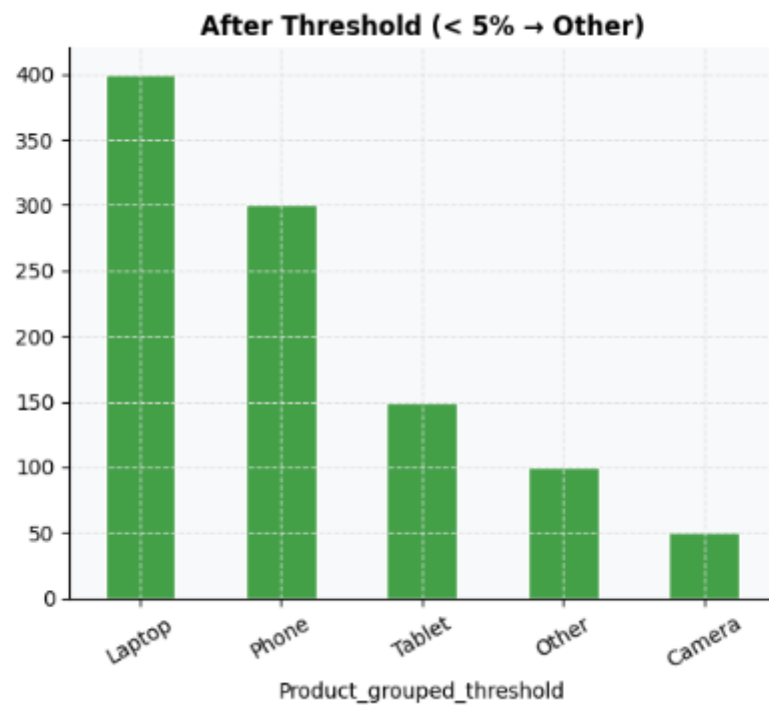
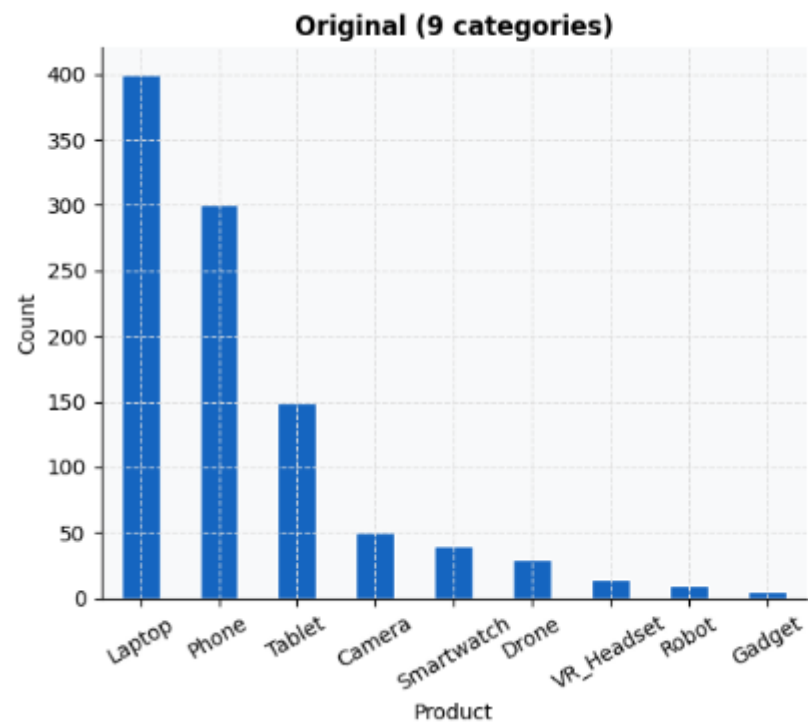
Total: 1000 rows

After Threshold Strategy:

Product_grouped_threshold	
Laptop	400
Phone	300
Tablet	150
Other	100
Camera	50

After Top-3 Strategy:

Product_top3	
Laptop	400
Phone	300
Tablet	150
Other	150





Topic 3: Discretization and Binning

Discretization converts **continuous numerical variables** into **categorical bins**.



Why Discretize?

Reason	Explanation
Reduce noise	Smooth out small fluctuations
Handle outliers	Extreme values grouped into edge bins
Capture non-linearity	Model non-linear relationships in linear models
Improve interpretability	"Age 26–35" is easier to understand than exact ages
Business requirements	Sales tiers, customer segments



Why NOT Discretize?

- **Information loss:** Ages 31 and 39 become identical if both fall in "30–40" bin
- **Complex models:** Tree-based models and neural networks already capture non-linearities — binning is unnecessary and may hurt performance
- **Small datasets:** Binning can make data too coarse



3.1 `pd.cut()` — Equal-Width Binning

Divides data into bins of **equal width** (or custom edges) based on the range of values.

Parameter	Description
<code>x</code>	Input array or Series to bin
<code>bins</code>	Integer → number of equal-width bins; or list → custom edges

```
# Without labels
age_bins_auto = pd.cut(ages, bins=4)
```

```
print('\nThe Unique Bins/Categories:')
print(age_bins_auto.cat.categories)
```

min = 5, max = 85, equal bin width = 20.0

`pd.cut()` with 4 equal-width bins (auto-labeled):

Age	Bin
5	(4.92, 25.0]
12	(4.92, 25.0]
17	(4.92, 25.0]
25	(4.92, 25.0]
34	(25.0, 45.0]
42	(25.0, 45.0]
51	(45.0, 65.0]
63	(45.0, 65.0]
72	(65.0, 85.0]
85	(65.0, 85.0]
20	(4.92, 25.0]
39	(25.0, 45.0]
58	(45.0, 65.0]

```
# With custom edges and labels
custom_bins = [0, 18, 35, 60, 100]
custom_labels = [
    'Youth (0-18)',
    'Young Adult (18-35)',
    'Middle-Age (35-60)',
    'Senior (60+)',
]
age_bins_named = pd.cut(
    ages, bins=custom_bins, labels=custom_labels, include_lowest=True
)
```

```
print('\nThe Unique Bins/Categories:')
print(age_bins_named.cat.categories)
```

pd.cut() with CUSTOM edges and labels:

Age	AgeGroup
5	Youth (0-18)
12	Youth (0-18)
17	Youth (0-18)
25	Young Adult (18-35)
34	Young Adult (18-35)
42	Middle-Age (35-60)
51	Middle-Age (35-60)
63	Senior (60+)
72	Senior (60+)
85	Senior (60+)
20	Young Adult (18-35)
39	Middle-Age (35-60)
58	Middle-Age (35-60)

The Unique Bins/Categories:

```
Index(['Youth (0-18)', 'Young Adult (18-35)', 'Middle-Age (35-60)',
      'Senior (60+)'],
      dtype='str')
```

- `age_bins_auto.cat.categories` — gets the unique bin interval labels from a categorical Series
- `.cat` — accessor for categorical dtype, used **only on Series** (like `.str` for strings, `.dt` for datetimes)
- `.categories` — attribute under `.cat` that returns the unique category labels
- `.cat` works on Series only, not DataFrames — access column as Series first: `df['col'].cat.categories` ✓ vs `df.cat.categories` ✗

```
salary_bins = pd.cut(
    salaries,
    bins=[0, 30000, 60000, 100000, 999999], # 4 custom bins (5 provided, so 4 bins)
    labels=[
        'Entry [$0K-$30K]',
        'Mid ($30K-$60K]',
        'Senior ($60K-$100K]',
        'Executive ($100K+]',
    ],
    include_lowest=True,
)
```

Salary Tier Classification:

Salary	Tier
18000	Entry [\$0K-\$30K]
30000	Entry [\$0K-\$30K]
60000	Mid (\$30K-\$60K]
45000	Mid (\$30K-\$60K]
0	Entry [\$0K-\$30K]
75000	Senior (\$60K-\$100K]
90000	Senior (\$60K-\$100K]
100000	Senior (\$60K-\$100K]
180000	Executive (\$100K+]

Tier distribution:

Entry [\$0K-\$30K]	3
Mid (\$30K-\$60K]	2
Senior (\$60K-\$100K]	3
Executive (\$100K+]	1

- By default `pd.cut` uses **(left, right]** intervals — meaning the **left edge is excluded**.
- `include_lowest=True` makes the **first bin's left edge inclusive** → `[0, 30000]` instead of `(0, 30000]`
- So without `include_lowest=True`, a salary of exactly `0` would be **NaN** (falls outside all bins). With it, `0` is included in the first bin.
- `salary_bins.value_counts()` — counts how many salaries fall in each bin
- `.sort_index()` — sorts by bin order instead of frequency, since `value_counts()` sorts by count descending by default
- `.to_string()` — prints without truncation



3.2 `pd.qcut()` — Equal-Frequency Binning (Quantile-based)

Creates bins with approximately **equal numbers of observations** by using quantiles.

Parameter	Description
<code>x</code>	Input data
<code>q</code>	Integer → number of quantiles (4=quartiles, 10=deciles); or list of quantile thresholds

```
qcut_bins = pd.qcut(data, q=4, precision=2, duplicates='drop')
```

```
cut_bins = [0, 25, 50, 75, 100]
cut_result = pd.cut(
    data,
    bins=cut_bins,
    labels=['Q1(0-25)', 'Q2(26-50)', 'Q3(51-75)', 'Q4(76-100)'],
    include_lowest=True,
)
```

```
# Quartile labels
quartiles = pd.qcut(
    income, q=4, labels=['Bottom 25%', 'Lower Mid 25%', 'Upper Mid 25%', 'Top 25%']
)
```

bin counts (pd.cut):

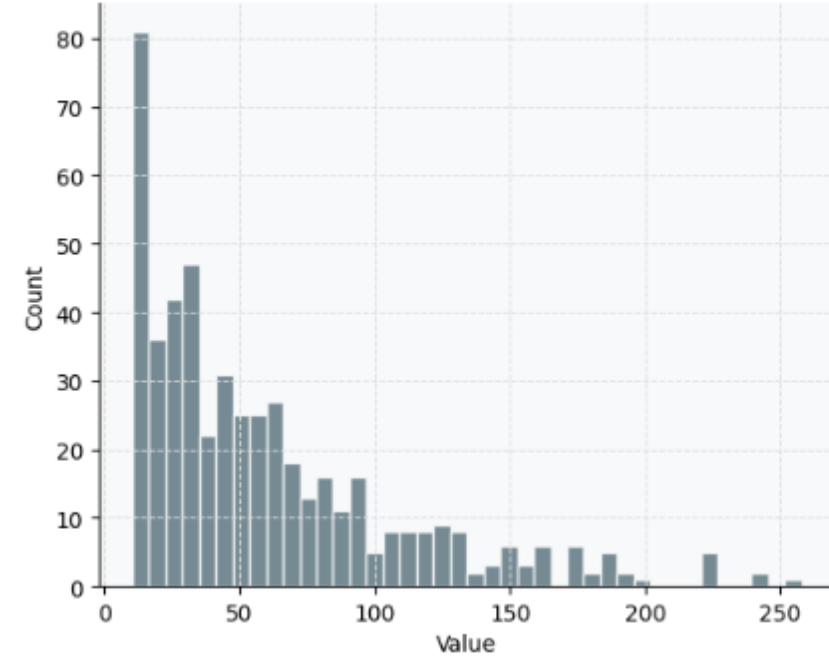
Q1(0-25)	2
Q2(26-50)	0
Q3(51-75)	5
Q4(76-100)	3

bin counts (pd.qcut – approx equal counts):

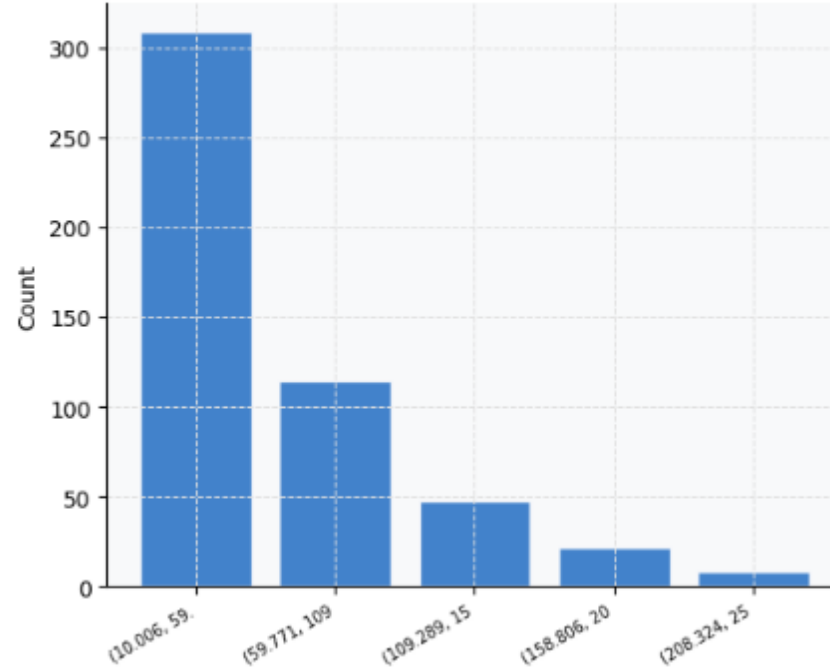
(14.99, 54.25]	3
(54.25, 73.5]	2
(73.5, 81.0]	2
(81.0, 93.0]	3

pd.cut() vs pd.qcut() on Skewed Data

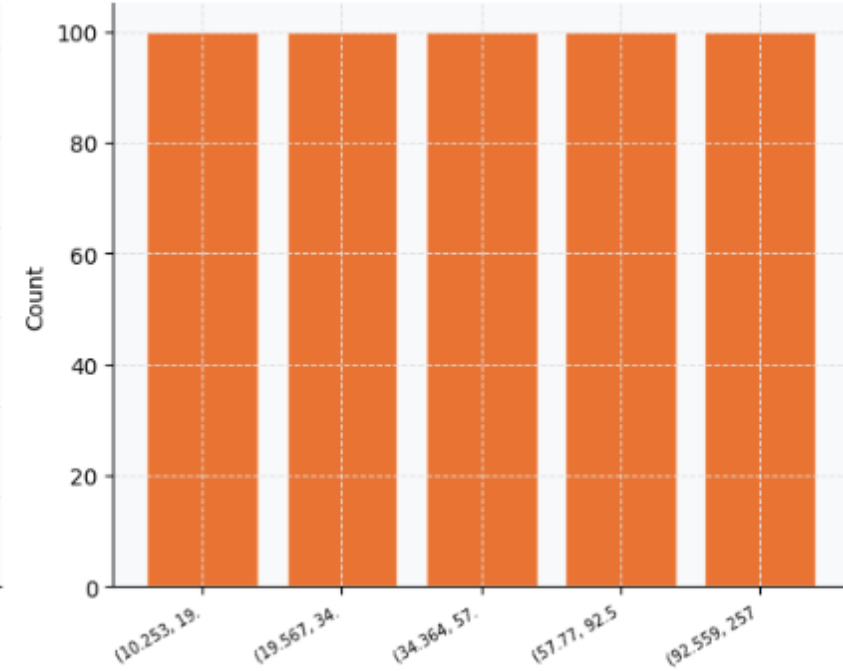
Original Skewed Data



pd.cut() — Equal Width (very unequal counts!)



pd.qcut() — Equal Frequency (balanced bins!)



- 💡 For skewed data: `pd.qcut()` creates MORE BALANCED bins than `pd.cut()`
Use `pd.cut()` when you need SPECIFIC ranges; `pd.qcut()` for balanced groups

Topic 4: Feature Interactions / Engineering

"Sometimes the relationship between features tells us more than the features themselves."

In machine learning, **capturing feature interactions** can dramatically improve model performance because they reveal patterns that individual features cannot express.

Why Feature Interactions Matter

Reason	Example
Non-Linearity	Income alone doesn't predict creditworthiness. $\text{Income} \times \text{Debt} \times \text{Credit History}$ together do.
Context Dependency	High BMI is more concerning for elderly than young people.
Signal Amplification	One late payment = minor issue. Late payment + High debt + Poor credit = major red flag.
Business Logic Encoding	Domain knowledge expressed as interactions (e.g., credit scores).
Model Performance	Linear models can't discover interactions automatically.

4.1 Arithmetic Combinations

4.1.1 Addition and Subtraction — Aggregates and Differences

Concept: Combine features to create meaningful aggregates or differences.

4.1.2 Multiplication and Division (Ratios)

Concept: Ratios normalize values and reveal relationships that raw numbers hide.

4.1.3 Complex Ratios and Proportions

Example 8: Sports Analytics - Player Efficiency

```
df = pd.DataFrame(  
    {  
        'player': ['Player_A', 'Player_B', 'Player_C', 'Player_D'],  
        'points': [1850, 1200, 2100, 1600],  
        'assists': [450, 320, 280, 520],  
        'rebounds': [380, 520, 310, 440],  
        'turnovers': [180, 140, 220, 160],  
        'minutes_played': [2400, 2100, 2600, 2300],  
    }  
)  
  
# Points per minute - efficiency  
df['points_per_minute'] = df['points'] / df['minutes_played']  
  
# Assist-to-turnover ratio - playmaking quality  
df['ast_to_ratio'] = df['assists'] / df['turnovers']  
  
# Usage rate approximation  
df['total_possessions'] = df['points'] + df['assists'] + df['rebounds']  
df['usage_rate'] = (df['total_possessions'] / df['minutes_played']) * 100  
  
# Efficiency rating (simplified)  
df['efficiency'] = (  
    df['points'] + df['assists'] + df['rebounds'] - df['turnovers']  
) / df['minutes_played']  
  
print(df[['player', 'points_per_minute', 'ast_to_ratio', 'efficiency']])
```

Example 8: Sports Analytics — Player Efficiency

- `player` — unique identifier for each player
- `points` — total points scored in the season
- `assists` — total assists (passes leading directly to a score)
- `rebounds` — total rebounds (recovering the ball after a missed shot)
- `turnovers` — total times the player lost possession to the opponent
- `minutes_played` — total minutes played in the season

Derived columns:

- `points_per_minute` — scoring rate → normalizes points by playing time for fair comparison
- `ast_to_ratio` (**Assist-to-Turnover Ratio**) — assists divided by turnovers → measures playmaking quality, higher is better (good playmakers create more than they lose)
- `total_possessions` — points + assists + rebounds → approximates how involved a player is
- `usage_rate` — % of team possessions a player is involved in per minute → measures player involvement
- `efficiency` — combined contributions (points + assists + rebounds) minus turnovers, per minute → simplified overall performance rating

	player	points_per_minute	ast_to_ratio	efficiency
0	Player_A	0.770833	2.500000	1.041667
1	Player_B	0.571429	2.285714	0.904762
2	Player_C	0.807692	1.272727	0.950000
3	Player_D	0.695652	3.250000	1.043478

Scouting Insights:

- Player_C scores most efficiently (0.81 pts/min) but has poor assist/turnover ratio (1.27)
- Player_D has excellent playmaking (3.25 ast/to) and highest overall efficiency
- Player_A balances scoring and playmaking well
- These ratios help evaluate player value beyond raw stats



4.2 Statistical Aggregations

4.2.1 Group-Based Features

Concept: Calculate statistics within groups to capture relative position and patterns.

```
# Group statistics
category_stats = (
    df_products.groupby('category')
    .agg(
        price_mean=('price', 'mean'),
        price_std=('price', 'std'),
        units_sold_mean=('units_sold', 'mean'),
        units_sold_std=('units_sold', 'std'),
        revenue_mean=('revenue', 'mean'),
    )
    .round(2)
    .reset_index()
)
```

- `groupby('category')` — splits df into 3 groups (Electronics, Clothing, Home)
- `.agg(price_mean=('price', 'mean'), ...)` — named aggregation syntax: `new_col_name=(source_col, aggregation_func)` creates one row per category with computed stats
- `.reset_index()` — moves `category` from index back to a regular column so it can be merged on

```
# Merge back
df_products = df_products.merge(category_stats, on='category', how='left')
print('\nAFTER MERGE: ')
print(df_products.to_string(index=False))
```

- `.merge(category_stats, on='category', how='left')` — joins category-level stats back to each product row matching on category name; `how='left'` keeps all products even if no match in category_stats

BEFORE MERGE:

Left:

product_id	category	price	units_sold	rating	revenue
1	Electronics	299	150	4.2	44850
2	Electronics	899	45	4.8	40455
3	Electronics	149	320	3.9	47680
4	Electronics	599	78	4.5	46722
5	Electronics	399	95	4.1	37905
6	Clothing	49	420	4.3	20580
7	Clothing	79	280	4.0	22120
8	Clothing	129	95	4.6	12255
9	Clothing	39	580	3.8	22620
10	Clothing	99	210	4.2	20790
11	Home	199	130	4.4	25870
12	Home	89	240	4.1	21360
13	Home	299	65	4.7	19435
14	Home	159	180	4.3	28620
15	Home	249	98	4.0	24402

Right:

	category	price_mean	price_std	units_sold_mean	units_sold_std	revenue_mean
	Clothing	79.0	36.74	317.0	188.33	19673.0
	Electronics	469.0	290.69	137.6	108.82	43522.4
	Home	199.0	80.93	142.6	69.02	23937.4

AFTER MERGE:

product_id	category	price	units_sold	rating	revenue	price_mean	price_std	units_sold_mean	units_sold_std	revenue_mean
1	Electronics	299	150	4.2	44850	469.0	290.69	137.6	108.82	43522.4
2	Electronics	899	45	4.8	40455	469.0	290.69	137.6	108.82	43522.4
3	Electronics	149	320	3.9	47680	469.0	290.69	137.6	108.82	43522.4
4	Electronics	599	78	4.5	46722	469.0	290.69	137.6	108.82	43522.4
5	Electronics	399	95	4.1	37905	469.0	290.69	137.6	108.82	43522.4
6	Clothing	49	420	4.3	20580	79.0	36.74	317.0	188.33	19673.0
7	Clothing	79	280	4.0	22120	79.0	36.74	317.0	188.33	19673.0
8	Clothing	129	95	4.6	12255	79.0	36.74	317.0	188.33	19673.0
9	Clothing	39	580	3.8	22620	79.0	36.74	317.0	188.33	19673.0
10	Clothing	99	210	4.2	20790	79.0	36.74	317.0	188.33	19673.0
11	Home	199	130	4.4	25870	199.0	80.93	142.6	69.02	23937.4
12	Home	89	240	4.1	21360	199.0	80.93	142.6	69.02	23937.4
13	Home	299	65	4.7	19435	199.0	80.93	142.6	69.02	23937.4
14	Home	159	180	4.3	28620	199.0	80.93	142.6	69.02	23937.4
15	Home	249	98	4.0	24402	199.0	80.93	142.6	69.02	23937.4

`groupby('category')`

- splits the DataFrame into groups based on unique values of `category`
 - does nothing on its own — must be chained with an aggregation like `.agg()`, `.mean()`, `.sum()` etc.
 - result has one row per unique category value
-

`.agg(new_col_name=('source_col', 'aggregation_func'))`

- named aggregation syntax — each argument defines one output column
 - `price_mean=('price', 'mean')` → create column `price_mean` = mean of `price` column
 - alternative syntax: `.agg({'price': 'mean', 'units_sold': 'std'})` → but this doesn't let you rename columns in the same step
 - available aggregation functions: `'mean'`, `'std'`, `'sum'`, `'count'`, `'min'`, `'max'`, `'median'`, `'first'`, `'last'`
-

`.merge(right_df, on='col', how='left')`

- joins two DataFrames on a common column
- `on='category'` — match rows where `category` value is equal in both DataFrames
- `how=` controls what rows to keep:
 - `'left'` — keep all rows from left df, fill NaN if no match in right
 - `'right'` — keep all rows from right df
 - `'inner'` — keep only rows that match in both (default)
 - `'outer'` — keep all rows from both, fill NaN where no match
- here `how='left'` ensures all 15 products are kept even if a category had no stats
- after merge, each product row gets the category-level columns added → enables row-level comparison against group averages

```
# Merge back
df_products = df_products.merge(category_stats, on='category', how='left')
print('\nAFTER MERGE: ')
print(df_products.to_string(index=False))
```

```
# Relative position within category
df_products['price_vs_category_mean'] = (
    df_products['price'] / df_products['price_mean']
).round(3)
df_products['sales_vs_category_mean'] = (
    df_products['units_sold'] / df_products['units_sold_mean']
).round(3)
df_products['sales_zscore_in_category'] = (
    (df_products['units_sold'] - df_products['units_sold_mean'])
    / df_products['units_sold_std']
).round(3)
```

```
# Performance flags
df_products['bestseller'] = (df_products['sales_zscore_in_category'] > 1.0).astype(int)
df_products['underperformer'] = (df_products['sales_zscore_in_category'] < -0.5).astype(int)
```


Product Analysis with Group Statistics:

product_id	category	price	units_sold	price_vs_category_mean	sales_vs_category_mean	bestseller	underperformer
1	Electronics	299	150	0.638	1.090	0	0
2	Electronics	899	45	1.917	0.327	0	1
3	Electronics	149	320	0.318	2.326	1	0
4	Electronics	599	78	1.277	0.567	0	1
5	Electronics	399	95	0.851	0.690	0	0
6	Clothing	49	420	0.620	1.325	0	0
7	Clothing	79	280	1.000	0.883	0	0
8	Clothing	129	95	1.633	0.300	0	1
9	Clothing	39	580	0.494	1.830	1	0
10	Clothing	99	210	1.253	0.662	0	1

4.3 Boolean and Logical Combinations

Concept: Combine conditions using AND/OR logic to create meaningful flags and risk scores.

Example 2: E-commerce Coupon Eligibility – complex AND/OR logic

```
df_coupons['is_vip'] = (  
    (df_coupons['total_spend'] >= 2000) & (df_coupons['total_orders'] >= 10)  
)<div data-bbox="26 487 123 512" data-label="Text">

# Coupon rules


```

df_coupons['is_new'] = (df_coupons['total_orders'] <= 2).astype(int)
df_coupons['is_dormant'] = (df_coupons['days_since_last_order'] > 90).astype(int)
df_coupons['is_engaged'] = (
 (df_coupons['newsletter_subscriber'] == 1) & (df_coupons['has_app'] == 1)
)<div data-bbox="26 685 350 969" data-label="Text">df_coupons['eligible_20_off'] = (
 (df_coupons['is_new'] == 1) & (df_coupons['newsletter_subscriber'] == 1)
)<div data-bbox="26 971 350 996" data-label="Text">df_coupons['eligible_vip_30'] = (
 (df_coupons['is_vip'] == 1) & (df_coupons['is_engaged'] == 1)
)<div data-bbox="26 998 350 1000" data-label="Text">df_coupons['eligible_50_off_200'] = (
 (df_coupons['total_spend'] >= 500)
 & (df_coupons['total_spend'] < 2000)
 & (df_coupons['days_since_last_order'] <= 60)
)<div data-bbox="26 1002 350 1000" data-label="Text">df_coupons['eligible_winback'] = (
 (df_coupons['is_dormant'] == 1)
 & (df_coupons['total_spend'] >= 1000)
 & (df_coupons['returned_item_ever'] == 0)
)<div data-bbox="26 1002 350 1000" data-label="Text">

E-Commerce Coupon Eligibility (Complex Boolean Logic):

customer_id	is_vip	is_dormant	is_engaged	best_offer
1	0	0	1	20% Off First Order
2	1	0	1	VIP 30% Off
3	0	1	0	No Offer
4	0	1	0	No Offer
5	1	0	1	VIP 30% Off
6	0	0	0	\$50 Off \$200
7	0	0	0	No Offer



Topic 5: Feature Selection

Feature selection is the process of **selecting a subset of relevant features** for model construction. It can dramatically impact:

-  **Model Performance**
-  **Interpretability**
-  **Computational Efficiency**

Hughes Phenomenon

Classifier performance **improves** with more features up to a point, but eventually **degrades** if the number of training samples is not increased proportionally.

Curse of Dimensionality

The phenomenon where algorithm performance deteriorates as the number of features increases. In high-dimensional spaces, distance becomes less meaningful and data becomes sparse.

Dimensions	Data Region Captured
1-D	~42%
2-D	~14%
3-D	~7%
4-D	~3%

Feature Selection Techniques

1. **Filter Methods** — Variance threshold, Correlation, Statistical Tests
2. **Wrapper Methods** — Forward Selection, Backward Elimination, RFE
3. **Permutation Importance** — Model-agnostic, post-hoc



5.1 Filter Methods

Filter methods evaluate features **independently of the model** using statistical measures. They are **fast** and **model-agnostic**.

5.1.1 Variance Threshold

Concept: Remove features with low variance (near-constant values provide little information).

```
from sklearn.feature_selection import VarianceThreshold
import numpy as np
```

```
# Apply variance threshold
selector = VarianceThreshold(threshold=0.01)
X = df_var.drop('target', axis=1)
X_selected = selector.fit_transform(X)
```

```
selected_features = X.columns[selector.get_support()].tolist()
removed_features = X.columns[~selector.get_support()].tolist()
```

- `.var()` — computes variance of each column → variance = how spread out values are; 0 = all values identical
- `.drop('target', axis=1)` — removes target column; `axis=1` means drop a column (`axis=0` would drop a row)
- `VarianceThreshold(threshold=0.01)` — creates a feature selector that removes any feature with variance ≤ 0.01
- `.fit_transform(X)` — `fit` computes variance of each feature, `transform` removes features below threshold → returns numpy array of remaining features
- `selector.get_support()` — returns a boolean array e.g. `[False, False, True, True]` → True means feature was kept
- `X.columns[selector.get_support()]` — uses boolean array to index column names → keeps only selected column names
- `X.columns[~selector.get_support()]` — `~` inverts the boolean array → gets removed column names

ORIGINAL:

	constant_feature	low_variance	medium_variance	high_variance
0	1	0	0	-0.877983
1	1	0	0	-0.826880
2	1	0	1	-0.226479
3	1	0	1	0.367366
4	1	0	1	0.913585
5	1	0	0	-0.803179
6	1	0	0	1.492689
7	1	0	1	-0.271124
8	1	0	0	-0.021367
9	1	0	0	-0.747212

Output of selector.fit_transform():

```
[[ 0.      -0.87798259]
 [ 0.      -0.82688035]
 [ 1.      -0.22647889]
 ...
 [ 0.       2.08844277]
 [ 0.      -1.58598331]
 [ 0.      -1.87992439]]
```

Output of selector.get_support():

```
[False False  True  True]
['constant_feature', 'low_variance', 'medium_variance', 'high_variance']
```

Selected features (2): ['medium_variance', 'high_variance']

Removed features (2): ['constant_feature', 'low_variance']

5.1.2 Correlation-Based Selection

Concept:

1. Remove features **highly correlated with each other** (multicollinearity)
2. Keep features **correlated with the target**

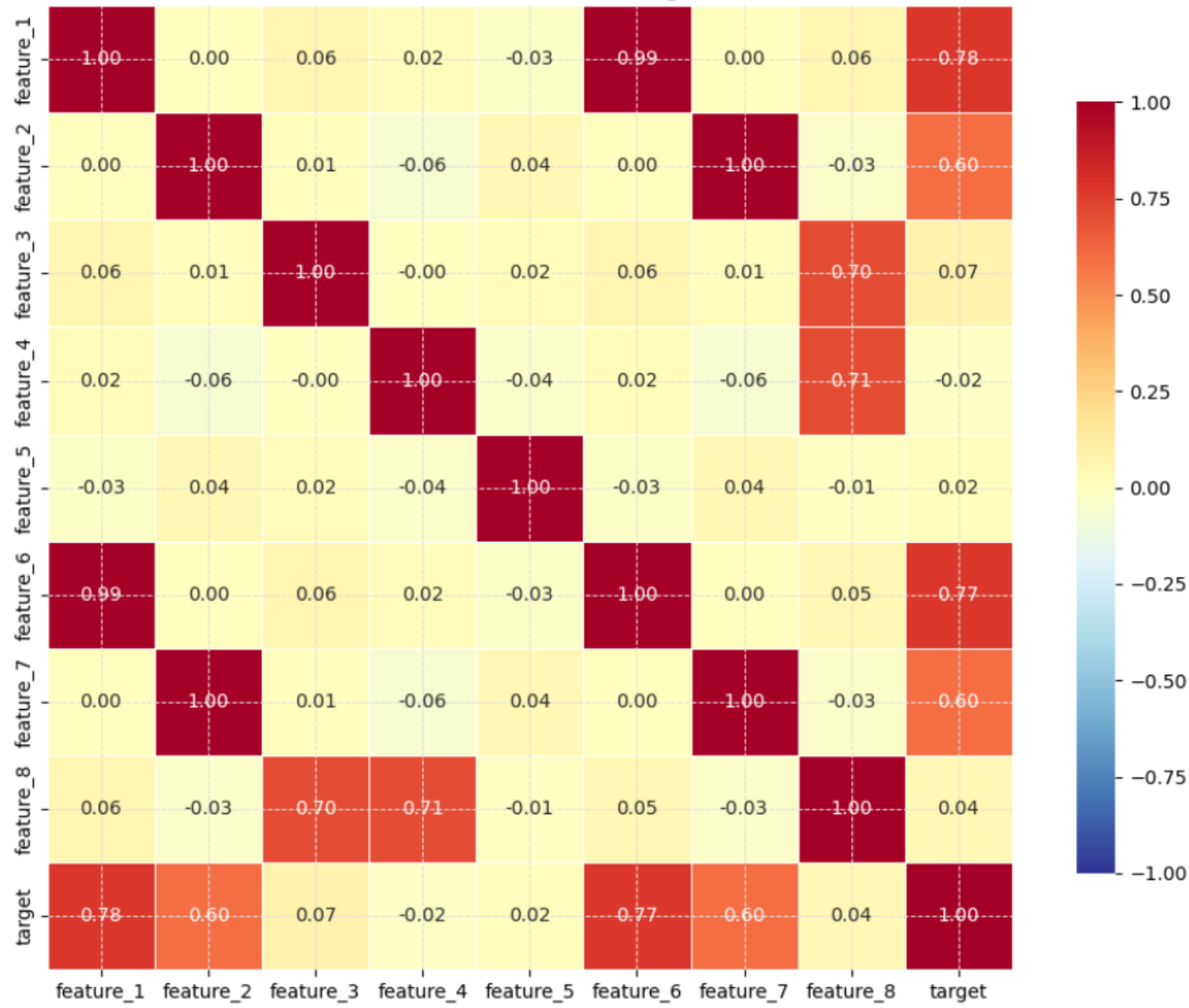
```
corr_matrix = df_corr.corr()  
print(corr_matrix)  
print("\nCorrelation with target (sorted):")  
print(corr_matrix['target'].sort_values(ascending=False).round(4).to_string())
```

Correlation with target (sorted):

target	1.0000
feature_1	0.7765
feature_6	0.7720
feature_2	0.6009
feature_7	0.5998
feature_3	0.0727
feature_8	0.0394
feature_5	0.0151
feature_4	-0.0168

- `.corr()` — computes pairwise Pearson correlation between all columns → returns a matrix of values between -1 and 1; 1 = perfect positive correlation, -1 = perfect negative, 0 = no linear relationship
- `corr_matrix['target']` — extracts just the column of correlations with target → shows how strongly each feature relates to what we want to predict
- `.sort_values(ascending=False)` — sorts from highest to lowest correlation → feature_1 and feature_2 and their correlated features feature_6 and feature_7 should appear at top since target was built from them

Correlation Matrix
(Red = Positive, Blue = Negative)




```
# Find and remove multicollinear features
def find_correlated_features(corr_matrix, threshold=0.9):
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    correlated_pairs = []
    for column in upper.columns:
        if column == 'target':
            continue
        corr_feats = upper.index[abs(upper[column]) > threshold].tolist()
        for cf in corr_feats:
            if cf != 'target':
                correlated_pairs.append(
                    {
                        'feature_A': column,
                        'feature_B': cf,
                        'correlation': round(corr_matrix.loc[column, cf], 3),
                    }
                )
    return pd.DataFrame(correlated_pairs)
```

```
find_correlated_features(corr_matrix, threshold=0.9)
```

- `np.triu(np.ones(corr_matrix.shape), k=1)` — creates an upper triangle matrix of 1s; `k=1` means start one above the diagonal (excludes diagonal which is always self-correlation = 1.0)
- `.astype(bool)` — converts 1s/0s to True/False mask
- `corr_matrix.where(...)` — keeps values where mask is True, sets rest to NaN → result is upper triangle only, avoiding duplicate pairs (e.g. feature_1 vs feature_6 appears once, not twice)

`where()` returns a **DataFrame** of the same shape as the original — values where condition is True are kept, everything else replaced with **NaN**.

```
corr_matrix.where(mask)
```

```
# same shape as corr_matrix
```

```
# True positions → original value kept
```

```
# False positions → NaN
```

- `for column in upper.columns` — iterates over each feature column
- `if column == 'target': continue` — skips the target column
- `abs(upper[column]) > threshold` — boolean mask of features that have correlation > 0.9 with current column
- `upper.index[...]` — gets the feature names that exceed the threshold
- appends a dict per correlated pair with both feature names and their correlation value
- returns a DataFrame of all highly correlated pairs

```

correlated_features = find_correlated_features(corr_matrix, threshold=0.9)
print("\nHighly correlated feature pairs (|r| > 0.9):")
print(correlated_features.to_string(index=False))

# Select features to remove
features_to_remove = set()
for _, row in correlated_features.iterrows():
    feat1, feat2 = row['feature_A'], row['feature_B']
    corr_1_t = abs(corr_matrix.loc[feat1, 'target'])
    corr_2_t = abs(corr_matrix.loc[feat2, 'target'])
    features_to_remove.add(feat2 if corr_1_t > corr_2_t else feat1)

features_to_keep = [
    c for c in df_corr.columns if c not in features_to_remove and c != 'target'
]

```

Highly correlated feature pairs (|r| > 0.9):

feature_A	feature_B	correlation
feature_6	feature_1	0.995
feature_7	feature_2	0.999

Features to REMOVE (multicollinear): ['feature_6', 'feature_7']

Selecting features to remove:

- `for _, row in correlated_features.iterrows()` — iterates over each correlated pair; `_` discards the row index
- `corr_matrix.loc[feat1, 'target']` — gets correlation of feat1 with target
- `corr_matrix.loc[feat2, 'target']` — gets correlation of feat2 with target
- `features_to_remove.add(feat2 if corr_1_t > corr_2_t else feat1)` — keeps the feature more correlated with target, removes the other
uses `set()` to avoid duplicates
- `[c for c in df_corr.columns if c not in features_to_remove and c != 'target']` — list comprehension keeping only non-removed, non-target columns



5.2 Wrapper Methods

Wrapper methods evaluate feature subsets by **training models and measuring performance**. They're slower but often more accurate than filter methods.

5.2.1 Forward Selection

Start with zero features, iteratively add the feature that improves validation performance the most.

When to stop? Performance plateau, or adding new features provides minimal improvement, or even worse performance.

```

def forward_selection(X, y, max_features=7):
    selected_features = []
    remaining_features = list(X.columns)
    results = []
    for iteration in range(max_features):
        best_score, best_feature = 0, None
        for feature in remaining_features:
            test_features = selected_features + [feature]
            model = LogisticRegression(random_state=42, max_iter=1000)
            scores = cross_val_score(
                model, X[test_features], y, cv=5, scoring='accuracy'
            )
            if scores.mean() > best_score:
                best_score = scores.mean()
                best_feature = feature
        if best_feature:
            selected_features.append(best_feature)
            remaining_features.remove(best_feature)
            results.append(
                {
                    'iteration': iteration + 1,
                    'added_feature': best_feature,
                    'cv_score': best_score,
                    'num_features': len(selected_features),
                }
            )
        print(
            f"Iteration {iteration+1}: Added '{best_feature}' (CV Score: {best_score:.4f})"
        )
    return pd.DataFrame(results), selected_features

```

```
results_df, selected_features = forward_selection(X_df, y_cls, max_features=7)
```

`forward_selection(X, y, max_features=7)`

- `selected_features = []` — starts empty, features added one by one
- `remaining_features = list(X.columns)` — all features available to pick from
- `outer for iteration in range(max_features)` — runs up to 7 iterations (one feature added per iteration)
- `inner for feature in remaining_features` — tries adding each remaining feature to the current selected set
- `selected_features + [feature]` — temporarily adds candidate feature to test it
- `LogisticRegression(random_state=42, max_iter=1000)` — logistic regression model; `max_iter=1000` increases max iterations to ensure convergence
- `cross_val_score(model, X[test_features], y, cv=5, scoring='accuracy')` — evaluates model using 5-fold cross validation; splits data into 5 folds, trains on 4, tests on 1, repeats 5 times → returns array of 5 scores
- `scores.mean()` — average accuracy across 5 folds → used to compare candidates
- `if scores.mean() > best_score` — keeps track of which candidate feature gave highest CV score
- `selected_features.append(best_feature)` — permanently adds the best feature found this iteration
- `remaining_features.remove(best_feature)` — removes it from candidates so it can't be picked again
- returns a DataFrame of results and the final selected feature list

Forward Selection Results:

iteration	added_feature	cv_score	num_features
1	feature_4	0.752	1
2	feature_5	0.812	2
3	feature_0	0.832	3
4	feature_2	0.854	4
5	feature_7	0.860	5
6	feature_6	0.860	6
7	feature_8	0.860	7

All 10 features: CV Score = 0.8540

Selected features: CV Score = 0.8600

5.2.2 Backward Elimination

Start with all features, iteratively remove the least useful feature.

Starting with all 10 features: CV Score = 0.8440

=====

Removed 'total_charges' → CV: 0.8470, 9 remain

Removed 'internet_service' → CV: 0.8460, 8 remain

Removed 'device_protection' → CV: 0.8470, 7 remain

Removed 'online_security' → CV: 0.8440, 6 remain

Removed 'payment_method' → CV: 0.8440, 5 remain

No feature can be removed without hurting score. Stopping.

Final features: ['tenure_months', 'monthly_charges', 'contract_type', 'tech_support', 'num_service_calls']

```

def backward_elimination(X, y, min_features=4):
    selected_features = list(X.columns)
    model = LogisticRegression(random_state=42, max_iter=1000)
    initial_score = cross_val_score(model, X, y, cv=5, scoring='accuracy').mean()
    print(
        f"Starting with all {len(selected_features)} features: CV Score = {initial_score:.4f}\n{'='*50}"
    )
    results = []
    while len(selected_features) > min_features:
        worst_score, worst_feature = initial_score, None
        for feature in selected_features:
            test_features = [f for f in selected_features if f != feature]
            m = LogisticRegression(random_state=42, max_iter=1000)
            score = cross_val_score(
                m, X[test_features], y, cv=5, scoring='accuracy'
            ).mean()
            if score >= worst_score:
                worst_score, worst_feature = score, feature
        if worst_feature:
            selected_features.remove(worst_feature)
            results.append(
                {
                    'removed_feature': worst_feature,
                    'cv_score': worst_score,
                    'remaining': len(selected_features),
                }
            )
            print(
                f"Removed '{worst_feature}' → CV: {worst_score:.4f}, {len(selected_features)} remain"
            )
        else:
            print("No feature can be removed without hurting score. Stopping.")
            break # ←
    return pd.DataFrame(results), selected_features

```

`backward_elimination(X, y, min_features=4):`

- starts with **all features** and removes one per iteration (opposite of forward selection)
- `initial_score` — baseline CV accuracy with all features
- `while len(selected_features) > min_features` — keeps removing until only 4 features remain
- inner loop tries **removing each feature one at a time** and scores the model without it
- `[f for f in selected_features if f != feature]` — list comprehension that creates feature list excluding the candidate for removal
- `if score >= worst_score` — if removing this feature doesn't hurt (or improves) the score → mark it as the worst (least useful) feature
- `selected_features.remove(worst_feature)` — permanently removes the least useful feature
- if no feature can be removed without hurting score, `worst_feature` stays None and loop ends early

Key difference from forward selection:

- forward selection: starts empty, greedily **adds** best feature each step
- backward elimination: starts full, greedily **removes** least useful feature each step
- backward is more expensive (starts with all features) but considers feature interactions from the start

5.2.3 Recursive Feature Elimination (RFE)

Concept: Train model → rank features by importance → remove weakest → repeat.

How importance is estimated:

- **Logistic Regression:** Absolute value of coefficients
- **SVM:** Weight vector coefficients
- **Random Forests / Tree-Based:** Feature importance (Gini impurity)

RFECV: Extension that automatically finds the optimal number of features using cross-validation.

```

from sklearn.feature_selection import RFE, RFECV
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rfe = RFE(estimator=rf, n_features_to_select=5, step=1)
rfe.fit(X_churn, y_churn)

feature_rankings = pd.DataFrame(
    {'feature': X_churn.columns, 'ranking': rfe.ranking_, 'selected': rfe.support_}
).sort_values('ranking')

print("RFE Feature Rankings:")
print(feature_rankings.to_string(index=False))
print(
    f"\nSelected features: {feature_rankings[feature_rankings['selected']][ 'feature'].tolist()}"

```

```

RFE Feature Rankings:
   feature  ranking  selected
tenure_months      1      True
monthly_charges     1      True
total_charges       1      True
contract_type       1      True
num_service_calls   1      True
tech_support        2     False
payment_method      3     False
internet_service    4     False
online_security     5     False
device_protection   6     False

```

```

Selected features: ['tenure_months', 'monthly_charges', 'total_charges', 'contract_type', 'num_service_calls']

```

- `RandomForestClassifier(n_estimators=100, random_state=42)` — creates a random forest model with 100 decision trees; `random_state=42` for reproducibility; used as the base estimator inside RFE to score feature importance
- `RFE(estimator=rf, n_features_to_select=5, step=1)` — Recursive Feature Elimination:
 - `estimator=rf` — the model used to evaluate feature importance at each step
 - `n_features_to_select=5` — keep only the 5 most important features
 - `step=1` — remove 1 feature per iteration (can be int for count or float for percentage)
- `rfe.fit(X_churn, y_churn)` — runs the RFE process:
 1. trains the model on all features
 2. ranks features by importance
 3. removes the least important feature
 4. repeats until only 5 features remain
- `rfe.ranking_` — array of integers, one per feature; `1` = selected (best), higher numbers = eliminated earlier (less important)
- `rfe.support_` — boolean array; `True` = feature was selected, `False` = eliminated

```
# RFECV – automatically finds optimal number
rfecv = RFECV(estimator=rf, step=1, cv=5, scoring='accuracy', min_features_to_select=3)
rfecv.fit(X_churn, y_churn)

print(f"Optimal number of features: {rfecv.n_features_}")
print(f"Optimal features: {X_churn.columns[rfecv.support_].tolist()}")
```

- `RFECV(estimator=rf, step=1, cv=5, scoring='accuracy', min_features_to_select=3)` — Recursive Feature Elimination with Cross Validation; same as RFE but automatically finds the optimal number of features instead of requiring you to specify it:
 - `estimator=rf` — random forest used to score feature importance
 - `step=1` — remove 1 feature per iteration
 - `cv=5` — 5-fold cross validation to evaluate each feature subset
 - `scoring='accuracy'` — metric used to compare feature subsets
 - `min_features_to_select=3` — never go below 3 features regardless of CV scores
- `rfecv.fit(X_churn, y_churn)` — runs the full RFECV process:
 1. starts with all features
 2. removes one feature per iteration
 3. evaluates each subset using 5-fold CV
 4. picks the subset size with the highest CV accuracy
- `rfecv.n_features_` — the optimal number of features found automatically by CV
- `rfecv.support_` — boolean array True/False per feature → True = in optimal subset
- `X_churn.columns[rfecv.support_].tolist()` — filters column names to only the selected ones

5.3 Permutation Importance

Shuffle feature values and measure the model's performance drop.

The drop in score compared to baseline = importance of that feature.

Model-agnostic — works with any model!

How It Works

1. Train model → record baseline score
2. **Shuffle** one feature's values (break its relationship with target)
3. Measure new score
4. Importance = baseline_score – shuffled_score
5. Repeat for each feature, `n_repeats` times for stability


```

from sklearn.inspection import permutation_importance
# Permutation importance
perm_imp = permutation_importance(
    rf_house, X_test, y_test, n_repeats=10, random_state=42
)

perm_results = pd.DataFrame(
    {
        'feature': X_houses.columns,
        'importance_mean': perm_imp.importances_mean,
        'importance_std': perm_imp.importances_std,
    }
).sort_values('importance_mean', ascending=False)

```

Permutation Importance (top features):

feature	importance_mean	importance_std
square_feet	1.8649	0.2307
bedrooms	0.0784	0.0104
lot_size	0.0275	0.0047
age_years	0.0085	0.0014
bathrooms	0.0052	0.0013
school_rating	0.0006	0.0006
hoa_fees	0.0004	0.0004
has_pool	0.0002	0.0003
distance_downtown	0.0002	0.0005
property_tax	0.0002	0.0006
crime_rate	-0.0001	0.0010
has_basement	-0.0002	0.0001
has_fireplace	-0.0004	0.0001
garage_spaces	-0.0011	0.0003

Permutation Importance Process — step by step:

1. compute baseline score on test set ($R^2 =$ e.g. 0.97)
2. **for each feature** (iterates over ALL features one by one):
 - randomly shuffle that feature's values → breaks its relationship with target
 - score the model on the shuffled data
 - importance = baseline score – shuffled score
 - repeat shuffle `n_repeats=10` times → get mean and std of importance
 - restore the feature to original values before moving to next feature
3. features with **high importance_mean** = big score drop when shuffled = truly important
4. features with **importance_mean $\approx$ 0** = shuffling them didn't hurt = useless features

Permutation Importance:

- `permutation_importance(rf_house, X_test, y_test, n_repeats=10, random_state=42)` — measures feature importance by:
 1. computing baseline score on test set
 2. randomly shuffling one feature's values (breaking its relationship with target)
 3. measuring how much the score drops → bigger drop = more important feature
 4. repeating `n_repeats=10` times per feature to get stable estimates
 - `n_repeats=10` — shuffles each feature 10 times → returns mean and std of importance
- `perm_imp.importances_mean` — average score drop across 10 repeats per feature
- `perm_imp.importances_std` — standard deviation of score drop → high std means unstable importance estimate
- `.sort_values('importance_mean', ascending=False)` — most important features appear first



Topic 6: Data Cleaning Log Report

"Data preparation is not just about writing code — it's about creating a reproducible, well-documented process."

Why Documentation Matters

Poor Documentation Leads To	Good Documentation Enables
Inability to reproduce results	Reproducibility
Uncertainty about transformations	Collaboration
Difficulty detecting errors	Auditing
Loss of institutional knowledge	Stakeholder trust

The Cleaning Report Has Three Sections

1. **Executive Summary** — one-paragraph overview with dataset name, record counts, processing time, key issues
2. **Data Quality Assessment** — initial state expressed quantitatively
3. **Transformation Log** — timestamped, ordered list of every action taken

DataCleaningLog Class:

`DataCleaningLog` is a logging utility class that tracks every transformation applied to a dataset during the data cleaning process.

Initialization: takes `dataset_name` and `analyst_name`, automatically records `start_time`, and sets up empty `log_entries` and `summary_stats`.

Core Functions:

`DataCleaningLog`

- `log_action()` → records a single cleaning step (category, action, rows/cols affected, before/after values)
- `record_initial_state()` → snapshots the dataset before cleaning (shape, dtypes, nulls, memory)
- `record_final_state()` → snapshots the dataset after cleaning
- `generate_summary()` → computes rows removed, columns removed, total actions, actions per category
- `to_dataframe()` → converts all log entries to a pandas DataFrame
- `to_dict()` → exports full log as a nested dictionary (metadata + states + actions)
- `save_to_json()` → saves `to_dict()` output as a JSON file
- `save_to_csv()` → saves `to_dataframe()` output as a CSV file
- `print_report()` → prints a human-readable executive summary to console

```
def __init__(self, dataset_name, analyst_name):
    self.dataset_name = dataset_name
    self.analyst_name = analyst_name
    self.start_time = datetime.now()
    self.log_entries = []
    self.summary_stats = {'initial': {}, 'final': {}}
```

```
def log_action(self, category, action, details,
               rows_affected=None, columns_affected=None, before_value=None, after_value=None,
               ):
    entry = {
        'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        'category': category,
        'action': action,
        'details': details,
        'rows_affected': rows_affected,
        'columns_affected': columns_affected if columns_affected else [],
        'before_value': str(before_value) if before_value is not None else None,
        'after_value': str(after_value) if after_value is not None else None,
    }
    self.log_entries.append(entry)
```

```
def record_initial_state(self, df):  
    """Record initial dataset statistics."""  
    self.summary_stats['initial'] = {  
        'total_rows': len(df),  
        'total_columns': len(df.columns),  
        'columns': list(df.columns),  
        'dtypes': df.dtypes.astype(str).to_dict(),  
        'missing_values': df.isnull().sum().to_dict(),  
        'memory_usage_mb': df.memory_usage(deep=True).sum() / 1024**2,  
    }
```

```
def record_final_state(self, df):  
    """Record final dataset statistics."""  
    self.summary_stats['final'] = {  
        'total_rows': len(df),  
        'total_columns': len(df.columns),  
        'columns': list(df.columns),  
        'dtypes': df.dtypes.astype(str).to_dict(),  
        'missing_values': df.isnull().sum().to_dict(),  
        'memory_usage_mb': df.memory_usage(deep=True).sum() / 1024**2,  
    }
```

```
# — Step 1: Create a messy dataset
```

```
# — Step 2: Initialize log and start cleaning
```

```
log = DataCleaningLog(  
    dataset_name="Customer_Churn_Dataset_2026", analyst_name="John Smith"  
)  
log.record_initial_state(df_raw)  
df = df_raw.copy()
```

```
# — STEP 1: Duplicates
```

```
print("\n[STEP 1] Checking for duplicate rows...")
```

```
dups = df.duplicated().sum()
```

```
if dups > 0:
```

```
    df = df.drop_duplicates()
```

```
    log.log_action(  
        category='Duplicates',  
        action='Removed duplicate rows',  
        details=f'Identified and removed {dups} exact duplicate rows. Kept first occurrence.',  
        rows_affected=dups,  
        columns_affected=list(df.columns),  
        before_value=f'Total rows: {dups + len(df)}',  
        after_value=f'Total rows: {len(df)}',  
    )
```

```

# — STEP 2: Missing Values
print("\n[STEP 2] Handling missing values...")
for col, method in [('age', 'median'), ('income', 'mean'), ('credit_score', 'median')]:
    missing_n = df[col].isnull().sum()
    if missing_n > 0:
        fill_val = df[col].median() if method == 'median' else df[col].mean()
        df[col] = df[col].fillna(fill_val)
        log.log_action(
            category='Missing Values',
            action=f'Imputed with {method}',
            details=f'{col} had {missing_n} missing values. Imputed with {method} = {fill_val:.2f}.',
            rows_affected=missing_n,
            columns_affected=[col],
            before_value='NaN',
            after_value=f'{fill_val:.2f}',
        )
    print(f"  ✓ {col}: imputed {missing_n} values with {method} ({fill_val:.2f})")

```

```

log.record_final_state(df)
print("\n" + "=" * 80)
log.print_report()

```


=====

DATA CLEANING REPORT – EXECUTIVE SUMMARY

=====

Dataset: Customer_Churn_Dataset_2026
Analyst: John Smith
Processing Date: 2026-03-23
Processing Time: 0:00:00.063829
Original Records: 1,020
Final Records: 1,003
Rows Removed: 17
Total Actions: 12

ACTIONS BY CATEGORY:

Duplicates	: 1 action(s)
Missing Values	: 3 action(s)
Outliers	: 2 action(s)
Invalid Values	: 2 action(s)
Data Standardization	: 1 action(s)
Text Cleaning	: 1 action(s)
Data Type Conversion	: 1 action(s)
Feature Engineering	: 1 action(s)

=====

```
# — Transformation Log DataFrame —————
log_df = log.to_dataframe()
print("Transformation Log:")
print(
    log_df[
        ['timestamp', 'category', 'action', 'rows_affected', 'columns_affected']
    ].to_string(index=False)
)
```

Transformation Log:

timestamp	category	action	rows_affected	columns_affected
2026-03-23 06:48:29	Duplicates	Removed duplicate rows	17.0	age, email, signup_date, city, state
2026-03-23 06:48:29	Missing Values	Imputed with median	50.0	[age]
2026-03-23 06:48:29	Missing Values	Imputed with mean	80.0	[income]
2026-03-23 06:48:29	Missing Values	Imputed with median	30.0	[credit_score]
2026-03-23 06:48:29	Outliers	Capped to valid range	10.0	[age]
2026-03-23 06:48:29	Outliers	Capped to valid range	5.0	[credit_score]
2026-03-23 06:48:29	Invalid Values	Set negative values to zero	14.0	[num_purchases]
2026-03-23 06:48:29	Invalid Values	Flagged invalid emails as NaN	25.0	[email]
2026-03-23 06:48:29	Data Standardization	Standardized placeholder values	NaN	[state]
2026-03-23 06:48:29	Text Cleaning	Standardized city names	NaN	[city]
2026-03-23 06:48:29	Data Type Conversion	Converted to integer	NaN	[age, credit_score]
2026-03-23 06:48:30	Feature Engineering	Created derived features	NaN	[account_age_days, income_category]